Category: Specification / Architecture Subcategory: Knowledge Infrastructure / Concurrency and Federation Document ID: MPLPB-SWARM-013 Last Updated: 2026-08-30T15:25Z v1.2 Owner: Mitchell D. McPhetridge, Independent Researcher Status: current Supersedes: —

MPLPB at Swarm and Enterprise Scale

Concurrent Writers, Federated Corpora, and the Ceiling on Amortization

Field	Value
Document ID	MPLPB-SWARM-013
Category	Specification / Architecture
Subcategory	Knowledge Infrastructure / Concurrency and Federation
Updated	2026-08-30T15:25Z v1.2
Owner	Mitchell D. McPhetridge, Independent Researcher
Status	current
Scope	What breaks when an MPLPB corpus is read and written by many agents at once, and when many corpora must be routed between under one authority; the write discipline, provenance class, and federation rules that contain those failures; the point at which the cost argument in MPLPB-COST-012 stops holding
When to use	Deploying MPLPB where more than one writer is active; running agents that write back to a corpus they also read; federating two or more corpora under shared routing; deciding whether swarm scale improves or degrades the economics
Supersedes	—

Related. Deployment Profiles (MPLPB-DEPLOY-015 v1 · current) — what a deployment may serve and write for four named audiences, and the measured reach-versus-precision trade. A Partial Ablation (MPLPB-TEST-014 v2 · current) — the run of §12.5's mechanical half, its four unmet requirements, and its negative result. MPLPB as a Local Web (MPLPB-LOCAL-008 v4 · current) — the corpus specification and the single-writer assumptions this paper drops. Smart Local MPLPB: A Retrieval-Bounded Front End for a Local Web (MPLPB-SMART-011 v1 · current) — the reader, its mode controller, and the teaching loop this paper puts under concurrency. MPLPB and the Cost of Visibility (MPLPB-COST-012 v2 · current) — the cost model whose amortization condition this paper bounds. MPLPB Separation & Precedence (MPLPB-SEP-007) — the declared-scope routing rule, applied here one level up.

Implementations. Swarm MPLPB — reference implementation of MPLPB-SWARM-013 v1; writer leases, supersession linearization, origin-depth gating, corpus registry, and health metrics. Standard library only, no network.

Back to. Main Index > Specification / Architecture Sub-Index

Abstract

Three prior documents specify a corpus, a reader, and an economics. All three assume one writer.

MPLPB-LOCAL-008 specifies a local web whose supersession discipline is described in the singular: a page is retired, a replacement is written, the log records the transition. MPLPB-SMART-011 specifies a front end that refuses to answer from anywhere except a retrieved page, and can be taught — a write path, exercised by one person at a keyboard. MPLPB-COST-012 argues that structural work at publication amortizes across many downstream retrievals, on the stated condition that the corpus is read far more often than it is revised.

Each assumption is reasonable at the scale it was written for. None survives a swarm.

This paper specifies what changes when many agents read and write one corpus concurrently, and when many corpora must be routed between under a single authority. It names ten failure modes, most of which are invisible to every check the prior specifications supply, and it specifies the write discipline, provenance class, and federation rules that make them detectable.

Its central mechanism is a provenance class. A reader that refuses to answer without retrieval stops one agent improvising. It does not stop a swarm, because an agent that improvises and then teaches the corpus converts its improvisation into a document with an identifier, a declared scope, a status, and correct provenance — after which a second agent retrieves it legitimately and the invention is indistinguishable from a finding. The corpus launders it. Validation cannot catch this, because validation reads structure and the structure is impeccable.

Its central admission is economic. The amortization argument in MPLPB-COST-012 §9 requires that reads greatly exceed revisions. A swarm scales reads and revisions together, so the ratio is invariant in the number of agents, and swarm scale does not improve the amortization — it enlarges both sides of it. Where taught writes require human ratification, the cost of that ratification does not amortize at all. The ceiling on this architecture is located at human attention, and this paper locates it rather than assuming it away.

What is not established here is whether laundering occurs at rates that matter, whether ratification survives volume, or whether any of this structure beats flat storage of the same files. That last one is the ablation named open in MPLPB-LOCAL-008 §14 and MPLPB-COST-012 §12.2. It is still open. Nothing in this paper should be read as though it had been run.

1. What Scale Changes

The prior specifications are correct within their declared scope and silently assume four things outside it.

One writer. Supersession in MPLPB-LOCAL-008 §11.5 is written as a sequence: retire, replace, log. A sequence has one actor. Two actors performing that sequence against the same document identifier produce two replacements, both valid, both current, both claiming to supersede the same predecessor.

Human-paced revision. MPLPB-COST-012 §9 introduces a maintenance term `m · C_m` and observes that the argument holds when `n >> m`. At human pace that inequality is comfortable — a person reads a reference page many times between revisions of it. An agent that writes back at machine pace does not obviously respect it.

One corpus root. The bounded traversal in MPLPB-LOCAL-008 §2.2 rejects links outside the declared root, which is what keeps a crawl finite and a corpus identifiable. It says nothing about what to do when a second root exists and a question could be answered from either.

A trustworthy writer. The teaching loop in MPLPB-SMART-011 §9 writes a validating, superseding, cited document. It validates structure. It does not and cannot validate that the person teaching it knew anything.

Dropping these four assumptions produces two distinct problems, and conflating them is the first mistake available.

---

2. Two Scales, Two Different Failures

Swarm scale and enterprise scale are usually discussed together and fail apart.

Swarm scale is a concurrency problem. Many agents, one organisation, one corpus, machine-paced reads and writes. What breaks is write ordering, identifier allocation, and the integrity of the provenance chain. Nothing about authority is contested; everything about sequence is.

Enterprise scale is a federation and authority problem. Many authors, many corpora, governance, classification, access control, audit. What breaks is routing between corpora with overlapping declared scopes, provenance across corpus boundaries, and the gap between a retrieval filter and an authorization decision. Concurrency is present but is not the hard part; the hard part is that two corpora both credibly claim a question.

The two compose badly if treated as one. A distributed lock does not tell you which corpus owns a question. A declared-scope precedence rule does not stop two agents writing the same identifier in the same millisecond. Sections 4 and 5 address the first. Sections 6 and 7 address the second. Section 9 addresses the cost consequence of both, which is where the paper's least comfortable result lives.

---

3. The Constraint Does Not Survive, and That Is Fine

MPLPB-COST-012 §3 declares a constraint: consumer hardware, consumer front ends, no author-operated retrieval infrastructure, no commercial visibility tooling, ordinary publication surfaces. The corpus described there was built under it and the paper calls it the central evidentiary asset.

At enterprise scale that constraint does not hold, and pretending otherwise would be the kind of quiet dishonesty this framework exists to make difficult.

An enterprise deployment needs determinism, freshness guarantees, access control, and low-latency retrieval over a large corpus. MPLPB-COST-012 §2.2 lists those four as properties the public form does not supply.

They are not oversights to be engineered around later; they are the reason the expensive stack exists. An organisation that needs them will operate infrastructure, and should.

What matters is being precise about what is lost and what carries over.

Lost. The cost result. MPLPB-COST-012's economic conclusion is a statement about an individual author's cost under a declared no-paid-infrastructure rule. It does not transfer to a deployment that runs a vector store, an identity provider, and a retrieval service. Anyone citing the cost paper in support of an enterprise MPLPB deployment is citing it outside its declared scope, which is precisely the failure the framework's precedence rule exists to prevent.

Carried over. The artifact discipline. Complete artifacts, declared identity and scope, enforced supersession, provenance that travels with the retrieved text, typed relationships, and a reader that refuses rather than confabulates. None of these depend on who pays for the index. They are properties of how the corpus is authored, and they remain available whether the retrieval layer is FTS5 in a laptop's memory or a managed cluster.

The constraint was an evidentiary device, not a virtue. Its job was to make a disclosure auditable by a stranger: run the implementation with the network off and see whether anything undisclosed is doing the work. That job is done by the reference implementations. Having done it, the architecture is free to be deployed on infrastructure that the constraint would have forbidden, provided nobody claims the constraint's evidence while doing so.

---

4. Concurrent Write Discipline

The swarm write problem is old and the solutions are boring. The correct move is to use a boring one and be explicit that it is boring, rather than to build something interesting that fails in ways nobody has characterised.

4.1 Identifier allocation

Document identifiers must be unique across a corpus. MPLPB-LOCAL-008 §11.3 makes duplicate identifiers a validator failure, which is correct and is a detection rather than a prevention. Two agents each choosing `OPS-NOTE-014` produce a validator failure after the fact, by which time both pages exist and one of them must be rewritten.

The rule is that identifiers are allocated, not chosen. An agent requests an identifier for a spoke and receives one; it does not construct one from a prefix and a count it read a moment ago. Allocation is serialised by the same lease that serialises writes, so the counter is only ever advanced by the current lease holder.

This is deliberately unclever. It costs one file write per allocation and removes an entire failure class.

4.2 Supersession must be linearizable per identifier

A supersession fork is two documents, both `status: current`, both naming the same predecessor in `mplpb:supersedes`. Both are structurally valid. The validator in MPLPB-LOCAL-008 §11 does not catch it, because nothing in the eight checks compares supersession claims across documents.

The corpus is now ambiguous about its own history, and the ambiguity is durable: a reader asking what replaced the predecessor gets two answers with equal standing, and no rule in the corpus decides between them.

The discipline is that supersession of a given identifier is a linearizable operation. Under a lease, a writer reads the current head for that identifier, writes its replacement, and advances the head. A writer whose lease expired between reading and writing is rejected on the write, not on the read — which is what a fencing token is for and why §4.3 has one.

Forks are still detectable after the fact, and the reference implementation checks for them, because a discipline with no detector is a hope.

4.3 Writer leases

A lease is a file. It names a spoke, an owner, an expiry timestamp, and a monotonically increasing fencing token. Acquiring a lease means writing that file when no unexpired lease exists. Writing to the spoke means presenting a token at least as large as the one the spoke last accepted.

The fencing token is the part that matters and the part usually omitted. Expiry alone is insufficient: a writer can acquire a lease, stall, have the lease expire, have a second writer acquire it, and then wake and complete a write it believed was still authorised. The token makes the stale write fail at the point of writing, because the spoke has already accepted a higher one.

Lease granularity is the spoke, not the document. A spoke is the unit of declared scope and therefore the unit an author reasons about; locking at document granularity would permit two agents to write the same spoke's index concurrently, which is where cross-document structure lives.

4.4 What is deliberately not built

No consensus protocol. No conflict-free replicated data types. No distributed transactions across corpora.

The justification is a claim about workload, and it should be stated as one so it can be wrong. Corpus writes are expected to be rare relative to reads even in a swarm, because a corpus that is written as often as it is read is not functioning as a reference — see §9, where that condition is exactly the economic failure case. A coarse lease with a fencing token is adequate for rare writes, is auditable by reading one file, and requires no service.

If a deployment finds that leases are contended enough to matter, that finding is more interesting than the lock: it means the corpus is being used as a message bus, and the architecture is the wrong one.

---

5. Epistemic Laundering

This is the failure the rest of the paper is arranged around.

5.1 The mechanism

MPLPB-SMART-011 §2 establishes an asymmetry: a front end that occasionally says *I don't have that* is mildly annoying, and one that occasionally invents a confident answer is corrosive. Every shipped mode

therefore carries `answer_without_retrieval: false`.

That constraint governs one reader answering one question. It does not govern what happens next, and MPLPB-SMART-011 §11 says so directly: the package can hand a model a retrieved page and decline to hand it anything else, but it cannot police what the model then says.

In a swarm there is a next step. Consider the sequence:

1. Agent A is asked something the corpus does not cover.
2. Agent A answers from its own prior knowledge — the model bypass named FM-L8 in MPLPB-LOCAL-008 §12.
3. Agent A, or a person relaying it, teaches the corpus that answer.
4. The teaching loop does its job correctly: the new page validates, carries a document identifier, a declared scope, a timestamp, a status, and a citation.
5. Agent B asks a related question, retrieves the new page, and answers from it — with provenance, under `answer_without_retrieval: false`, entirely within the rules.

At step 5 the corpus is being consulted properly and the answer is invented. Every structural check passes. The provenance is accurate: the page really does exist, really is current, really is local, really says what it is quoted as saying. What the provenance does not record is that its content was never retrieved from anything.

Call this laundering, because that is what it is: the corpus converts an unsourced assertion into a sourced one by the ordinary operation of its own discipline. The discipline is not malfunctioning. It is functioning, on input it cannot inspect.

5.2 Why validation cannot fix it

The eight structural checks in MPLPB-LOCAL-008 §11 read structure. Laundering produces impeccable structure. There is no check over well-formedness that distinguishes a taught page recording a genuine finding from a taught page recording a confabulation, because the difference is in the world and not in the file.

MPLPB-LOCAL-008 §14 Test 3 already makes the parallel point in a different register: field utilization detects whether metadata is *consulted*, not whether consulting it *helps*. Here the gap is one step wider — structural validity detects whether a page is *well-formed*, not whether it is *sourced*.

Any proposal to fix this by inspecting content should be refused. A validator that judged whether a claim was true would be a truth engine, and MPLPB-COST-012 §7 spends a section refusing to be one. The janitor and the librarian do not adjudicate.

5.3 The provenance class

What is available is to make the condition visible and to bound its accumulation. Two fields:

`mplpb:origin` — one of `human`, `taught`, `derived`.

`mplpb:origin_depth` — an integer. `0` for a page a human authored or ratified. For a page taught from retrieved material, one greater than the maximum depth of the pages cited in its own creation.

The depth field is the load-bearing one. A taught page grounded in human-authored sources has depth 1. A page taught from that page has depth 2. A chain of agents teaching each other produces a monotonically climbing integer, and the climb is the signal.

Three rules follow:

Retrieval does not hide depth. A hit at depth greater than 0 carries its depth in the citation, exactly as MPLPB-SMART-011 §6 makes `substrate` inseparable from the hit object rather than a render-time decoration. The same reasoning applies for the same reason: a taught note cited as an authored one reads better than the truth, which is what makes that failure quiet.

Depth beyond a threshold requires ratification. The reference implementation defaults the threshold to 1: a page may be taught from human-authored material without ceremony, and a page taught from taught material is refused unless a human ratifies it. Ratification writes `origin: human`, `origin_depth: 0`, and a ratifier identity. It is a signature, not a compliment.

The threshold is configuration, not doctrine. A deployment that sets it to 3 has decided to accept three generations of machine authorship between human checks. That is a legitimate choice and the number should be visible in the corpus rather than buried in a service, so that someone auditing later can see what was permitted.

5.4 What this does not do

It does not detect laundering. It bounds how far an unratified chain can run before a human is asked, and it makes the resulting depth visible to anything that retrieves.

A human who ratifies without reading defeats it completely. That failure has its own entry — FM-S9 in §11 — and its own detector, which measures ratification latency rather than ratification quality, because latency is observable and quality is not. A ratification population whose median latency is four seconds is not ratifying.

This is a weaker result than it would be comfortable to claim, and it is the honest one. The mechanism converts an invisible failure into a visible number. Whether anyone looks at the number is outside the corpus.

---

6. Federation

Enterprise deployments have more than one corpus, for reasons that are usually organisational rather than technical: different owners, different classifications, different retention rules, different release cadences.

6.1 Roots stay bounded

The bounded traversal in MPLPB-LOCAL-008 §2.2 is unchanged. A crawl starting at a root resolves relative links, rejects anything outside that root, and visits each reachable page once. Federation does not relax this. A link from corpus A to corpus B is an external link and is treated as one.

This is the property that keeps a corpus identifiable. A corpus whose crawl can wander into another corpus has no boundary, and a thing with no boundary cannot declare a scope.

6.2 The registry is a corpus

Routing across corpora needs a directory: which corpora exist, where their roots are, what each declares as its scope, who owns it, and what substrate it lives on.

That directory is itself an MPLPB corpus — HTML pages with metadata blocks, one page per registered corpus, an index at the root. This is not a decorative symmetry. It means the registry validates with the same validator, supersedes with the same discipline, carries provenance with the same fields, and is crawled by the same crawler. A registry maintained as a YAML file in a service's configuration is a second kind of object with a second set of failure modes and no janitor.

6.3 Cross-corpus routing is the precedence rule, one level up

MPLPB-SEP-007 decides between spokes by declared scope, and returns `ambiguous` when ownership cannot be resolved past the margin — naming the spokes touched, quoting each declared scope, and asking the user to narrow. MPLPB-COST-012 §7 restates why: two domains that always defer to whoever spoke last are not separate, they are one blurred instance wearing two names.

Cross-corpus routing applies the identical rule with corpora in place of spokes. A query is scored against each registered corpus's declared scope; a clear winner is routed to; a tie within the margin returns `ambiguous`, names the corpora, quotes their declared scopes, and stops.

It stops rather than merging. Merging results from two corpora with different owners, classifications, and retention rules produces an answer that no single owner is accountable for, and accountability is most of what an enterprise deployment is buying.

6.4 Provenance across the boundary

A citation of a page in another corpus carries the corpus identifier alongside the fields MPLPB-SMART-011 §6 already requires:

```
[CORPUS-B · POL-RETENTION-004 · policy/retention.html · v3 · local · current · origin=hu
    man d0]
```

The format is longer than anyone wants. That is the same deliberate awkwardness as the `substrate` field: there is no shorter form, so the honest version is the default. Dropping the corpus identifier while keeping the document identifier produces a citation that looks local and is not, which is FM-L10 reappearing one level up and is entered as FM-S10.

---

7. Access Control, Declined

MPLPB-SMART-011 states that `mplpb:protected` filters retrieval and is not authentication, and declines to implement an inference-based gate on the grounds that dressing a stated caveat as a mechanism is worse than the caveat.

That position is correct and this paper does not improve on it. It is worth restating precisely because enterprise scale is exactly where the temptation to improve on it arrives.

Authentication and authorization belong to the substrate beneath the corpus: filesystem permissions, repository access, an identity provider, a network boundary. These are solved problems with mature implementations, none of which are improved by being reimplemented in a crawler.

What the corpus layer adds is narrow and worth having:

- A page declares its classification as a field, so the classification travels with the page rather than living in a separate access-control list that drifts from it.
- A retrieval result records which classification was used to decide it was returnable, so an audit can reconstruct the decision.
- The filter is named a filter everywhere it appears, so that no reader mistakes a retrieval exclusion for an access denial.

The distinction is not pedantic. A filter that quietly omits a page produces the same user experience as a corpus that lacks it, and an operator who believes the filter is a gate will eventually place material behind it that needed a gate. Naming it is the mechanism.

---

8. Corpus Health as a Measured Quantity

At one author's scale, corpus condition is assessed by looking. At swarm scale nobody looks, so the condition has to be a number or it is nothing.

Five quantities, all computable from crawl records with no network and no service:

Metric	Definition	What a rise means
Ambiguity rate	Routing decisions returning `ambiguous` ÷ total decisions	Declared scopes are overlapping; spokes or corpora need re-drawing
Supersession forks	Count of identifiers with more than one current successor	Write discipline is being bypassed or leases are being ignored
Depth histogram	Distribution of `origin_depth` across current pages	Machine authorship is accumulating between human checks
Retired mismatch	Indexed `status: retired` records ÷ files under `_log/superseded/`	The retired ingestion path of MPLPB-LOCAL-008 §7.1 is not running
Maintenance rate	Write and supersession events per unit time, against read events	The `m` term of MPLPB-COST-012 §9, finally measured rather than assumed

Thresholds are not supplied. A deployment's tolerable ambiguity rate depends on how finely it has drawn its spokes, and inventing a number here would be inventing an authority the corpus does not have. What is supplied is the measurement, on the argument that a quantity nobody computes cannot be a quantity anybody manages.

The last row is the one this paper treats as most important, and §9 explains why.

---

9. The Amortization Ceiling

MPLPB-COST-012 §9 gives the cost model. An author pays a structural cost `C_s` once, maintenance `C_m` over `m` events, and downstream readers pay a reduced reconstruction cost `C'_r` over `n` retrievals:

```
C_s + m · C_m + n · C'_r     versus     n · C_r
```

The paper states the condition honestly: this is smaller when the structural work substantially reduces per-retrieval reconstruction **and** `n >> m`. It also names maintenance as the term most likely to grow with corpus size, and pruning as the part not yet mechanized.

Swarm scale attacks that condition directly, and in two ways.

9.1 Scaling both sides

Let `a` be the number of agents. Reads scale with `a` — that is the point of a swarm. But agents that write back also scale writes with `a`:

```
n = a · n■          m = a · m■          n / m = n■ / m■
```

The ratio is invariant in `a`. Adding agents does not improve amortization; it enlarges both sides of the comparison by the same factor. The structural work still amortizes exactly as well as it did with one agent, and no better.

This is a mildly disappointing result rather than a fatal one. It says swarm scale is neutral on the economics, not that it is harmful. The harmful part is next.

9.2 The term that does not amortize

Let `r` be the fraction of taught writes requiring human ratification under §5.3, and `C_h` the cost of a ratification. Total cost becomes:

```
C_s + m · C_m + r · m · C_h + n · C'_r
```

`C_m` is machine cost and falls with tooling. `C_h` is human attention and does not. As `a` grows, `m` grows, and `r · m · C_h` grows linearly with the number of agents while being paid in a currency that does not scale.

This is the ceiling, and it is worth being exact about where it sits. It is not at compute, storage, index size, or corpus size. It is at the number of judgements a human population can make per unit time. A deployment can add agents until ratification saturates, and then adding agents makes the corpus worse rather than larger, because the marginal write either waits or gets ratified without being read — which is FM-S9.

9.3 The condition under which swarm MPLPB is worth it

Stating it plainly, because it is the operative question:

Swarm MPLPB is economical where agents read the corpus far more often than they write to it. A swarm of readers over a human-maintained corpus is the good case, and it is a common one — many agents consulting shared reference material, none of them authoring it.

A swarm that writes as often as it reads is not using a knowledge corpus. It is using a distributed log with metadata attached, and it will pay corpus prices for log behaviour. The right response to discovering this is architectural, not economic: the material being written at that rate is event data and belongs somewhere event data belongs.

This is the least comfortable result in the paper and it is placed here rather than in a footnote for that reason. The prior paper's amortization argument was already conditional; this one locates the condition's boundary and finds that a swarm can cross it.

---

10. Who This Is For

Swarms of readers over shared reference material. Many agents, one well-maintained corpus, writes rare and human-originated. The concurrency machinery in §4 is nearly idle, the depth histogram in §8 stays flat, and the amortization in §9 works as the prior paper describes.

Organisations federating a small number of owned corpora. Two to twenty roots with genuinely distinct owners and declared scopes, routed by §6. The registry is small enough that a person can read it, which is the condition under which declared scopes stay honest.

Deployments that have already decided to operate infrastructure. §3 gives up the cost result explicitly. What remains is the artifact discipline, which is worth having on a managed stack and is not what a managed stack supplies.

Not swarms whose primary output is corpus writes. See §9.3. The architecture will function and the economics will not.

Not deployments needing the corpus layer to enforce access. See §7. The filter is a filter.

Not anyone who needs the ablation to have been run. See §12.5. It has not.

Settings for each of these are specified separately in MPLPB-DEPLOY-015, which adds two audiences this section did not anticipate — an internal assistant answering staff, and an external one answering customers. The second is where the provenance machinery in §5 stops being bookkeeping: an organisation can state that no machine-authored page reaches a customer, and `origin_depth` is what makes that checkable rather than aspirational.

---

11. Failure Mode Index

The FM-L series in MPLPB-LOCAL-008 §12 covers the corpus. This series covers what concurrency and federation add. Numbering is independent; an FM-S identifier never refers to an FM-L failure.

FM-S1 — Identifier collision. Two writers allocate the same document identifier. *Detection:* the duplicate-identifier check in MPLPB-LOCAL-008 §11.3 catches it after both pages exist. *Prevention:*

allocation under lease, §4.1.

FM-S2 — Supersession fork. Two documents, both current, both naming the same predecessor. *Detection:* group current pages by their `mplpb:supersedes` entries and report any predecessor with more than one successor. Not covered by the eight structural checks.

FM-S3 — Epistemic laundering. An unsourced assertion enters the corpus through the teaching loop and is subsequently retrieved as sourced material. *Detection:* not directly detectable. `origin_depth` bounds the chain length and exposes it in citations; the depth histogram in §8 shows accumulation. This is a containment, not a detection, and §5.4 says so.

FM-S4 — Stale-lease write. A writer completes a write after its lease expired and was reacquired by another. *Detection:* the receiving spoke rejects a fencing token lower than the last accepted one, and the rejection is logged. A deployment with no such rejections in its log has either no contention or no fencing.

FM-S5 — Cross-corpus scope collision. Two registered corpora declare scopes that overlap enough that routing cannot resolve a class of queries. *Detection:* the ambiguity rate in §8, partitioned by corpus pair. A pair appearing repeatedly is a boundary that needs re-drawing, not a router that needs tuning.

FM-S6 — Registry drift. The registry names a corpus root that no longer exists, no longer validates, or has changed its declared scope without the registry entry being superseded. *Detection:* validate every registered root on a schedule and compare each root's declared scope against its registry entry.

FM-S7 — Filter/authorization confusion. An operator places material behind `mplpb:protected` believing it to be an access control. *Detection:* not mechanically detectable, which is why §7 makes naming it the mechanism. The nearest available check is whether any page's classification field claims a level the substrate beneath it does not enforce.

FM-S8 — Maintenance saturation. Write events approach read events; the corpus is being used as a log. *Detection:* the maintenance rate in §8. This is the FM entry for the §9.3 condition and it is the one most likely to be dismissed as a scaling success.

FM-S9 — Ratification theatre. Humans ratify without reading, and the depth gate becomes a formality. *Detection:* the distribution of ratification latency and the ratio of ratifications to rejections. A population that never rejects is not deciding. Latency is a proxy for attention and a poor one; it is used because it is observable and quality is not.

FM-S10 — Federated provenance loss. A cross-corpus citation drops the corpus identifier, so a page from another corpus is cited as though it were local to the reader's own. *Detection:* citations are generated from the hit object and the corpus identifier is a field of it, so a citation lacking one indicates a render path that constructed the string itself. This is FM-L10 one level up and has the same cause.

---

12. Falsifiers

Each would count against the argument if it came back negative. The first four are run by the reference implementation. The last three are open.

12.1 — Concurrency. Run many writers against one corpus with leases and fencing enabled. If identifier collisions or supersession forks occur, the write discipline in §4 does not do what it claims.

12.2 — Laundering containment. Seed an invented claim at depth 0 through a bypassing agent, then attempt to teach a second page from it. If the second page is written without ratification, or if either page is retrievable without its depth appearing in the citation, the mechanism in §5.3 is decorative.

12.3 — Federation boundary. Crawl a registered corpus whose pages link into a second registered corpus. If any page outside the crawled root enters the index, the boundary in §6.1 has failed and neither corpus has a scope.

12.4 — Ambiguity honesty. Issue a query answerable from two corpora with overlapping declared scopes. If the router returns a merged answer rather than `ambiguous` with both scopes quoted, §6.3 has collapsed into the behaviour MPLPB-SEP-007 exists to prevent.

12.5 — The ablation (open). Unchanged from MPLPB-LOCAL-008 §14 Test 4 and MPLPB-COST-012 §12.2. Duplicate the corpus, strip metadata, indexes, and typed links, flatten and randomize it, score both copies on the same probe set in randomized order with a blind scorer. If the stripped copy scores the same, the structure is decoration. This has been open across three specifications and is not closed by a fourth. Until it is run, the correct statement remains that structure has not been shown to beat storage.

Partial runs of the mechanical half are logged as MPLPB-TEST-014 v2. They fail four of this test's requirements — no blind scorer, no naive probe author, retrieval scored rather than reconstruction, and a synthetic corpus — so they do not close §12.5. The first run is withdrawn as uninterpretable: its control arm sat at 100%, so it could detect structure hurting and never structure helping. The second, after grading for a confound its own pre-registration caught, found that declared trigger text makes pages retrievable by vocabulary absent from their prose, surviving dilution to one relevant word in three at 79.2% against 0.0%. It also found that this advantage vanishes entirely under a prose-only broad fallback, which is now a per-deployment setting rather than a default. Anyone citing this section should read that ledger rather than this paragraph.

12.6 — Ratification decay (open). Measure whether ratification quality falls as ratification volume rises. Requires a seeded population of pages containing known errors, ratified by people who do not know which. If quality falls with volume, the depth gate in §5.3 is load-bearing only at low volume, and §9.2's ceiling is lower than the arithmetic suggests.

12.7 — Laundering incidence (open). Measure how often FM-S3 actually occurs in a running deployment, by sampling taught pages and checking their claims against sources independently. The mechanism in §5 is justified by the failure being severe, not by evidence that it is frequent. If it is rare, the depth gate is overhead. Nothing here establishes the rate.

---

13. What This Does and Does Not Establish

It does not establish:

- that MPLPB scales to arbitrary numbers of agents;
- that the cost result in MPLPB-COST-012 transfers to enterprise deployment — §3 gives it up explicitly;
- that epistemic laundering occurs at a rate that matters — see §12.7;
- that human ratification survives volume — see §12.6;
- that declared-scope routing between corpora outperforms a merged index;
- that structure beats flat storage of the same files — see §12.5, open since MPLPB-LOCAL-008. MPLPB-TEST-014 v2 found a retrieval advantage that survives vocabulary dilution, on a synthetic corpus with author-written probes. That is a direction, not a result, and the residual confound is named in its §5.

It does establish, at the strength of a reference implementation that can be executed:

- that the concurrent write failures in §4 are preventable by boring means, and that the prevention is auditable by reading one lease file;
- that supersession forks are detectable, though not caught by any check the prior specifications supply;
- that a provenance depth counter bounds unratified machine authorship and makes its accumulation visible in citations and in a histogram;
- that federated routing can preserve corpus boundaries and return `ambiguous` rather than merging across owners.

The evidence class for the swarm claims is implementation, not deployment. No corpus described here has been run by a real organisation with real agents under real load. Everything in §8 and §9 is arithmetic and instrumentation awaiting a deployment to measure. That is a weaker class than the prior papers' author-run trials, because those at least involved apparatus.

Existence, comparability, superiority, and universality remain four different claims. This paper argues existence for the mechanisms, declines the rest, and adds a fifth thing it does not claim: that the mechanisms are necessary at any particular scale.

---

14. Conclusion

The prior specifications describe a corpus that one person writes and many machines read. That arrangement has properties worth keeping and assumptions worth naming.

Adding writers breaks ordering, and ordering is repaired by leases and fencing tokens that nobody will find interesting. Adding corpora breaks routing, and routing is repaired by applying the precedence rule one level up and refusing to merge across owners. Both repairs are unremarkable, which is the intended outcome.

The interesting failure is the one that does not look like a failure. A swarm that teaches itself produces a corpus in perfect structural health whose contents are increasingly its own inventions, correctly cited, properly superseded, fully provenanced, and unsourced. No validator will object. The only available responses are to count the generations, expose the count in every citation, and require a human somewhere in the chain —

knowing that the human is the ceiling, that the ceiling is low, and that a human who ratifies without reading removes it entirely.

The economics follow from that. Swarm scale does not improve amortization, because it scales reads and writes together. What it can do is push a corpus past the condition under which amortization held at all, into a regime where the thing being maintained is not a reference but a log. That boundary is now locatable, and locating it is most of what this paper is for.

Write complete artifacts. Allocate their identifiers rather than choosing them. Order their supersessions. Record where their content came from and how many machines it passed through on the way. Route between them by declared scope and refuse to merge across owners.

Then count how often anyone reads them, and how often anyone writes them, and check that the first number is much larger than the second.

Philosophical Note

A library survives being read.

Whether it survives being written by its readers is a different question, and the answer depends entirely on how many of them there are and how carefully anyone is watching.

The structure does not care. That is its virtue and its whole limitation.

— MDM

Source note

All citations to the corpus specification refer to MPLPB-LOCAL-008 v4 · current. All citations to the front end refer to MPLPB-SMART-011 v1 · current. All citations to the cost analysis refer to MPLPB-COST-012 v2 · current. The precedence rule is cited as MPLPB-SEP-007; that document carries no version in the source table available at the time of writing and is cited by identifier only.

The reference implementation accompanying this paper is released as MIT-licensed code with CC BY 4.0 documentation. Its test suite runs on the standard library with no network. The build tool under `tools/` requires `reportlab` to render this paper as PDF; that is a documentation dependency, is not imported by the package, and is not required to run the implementation or its tests. It is disclosed here because a claim about a dependency surface should include the parts of the repository that are inconvenient for it.

Directory names in the reference implementation are lowercase throughout, and the commands in its README match the directories on disk exactly. This is recorded because the two prior reference implementations ship capitalised directories and document lowercase ones, which makes their test suites unrunnable as written on case-sensitive filesystems — a defect found by executing the audit described in MPLPB-COST-012 §12.1

rather than by reading the code. That audit is the mechanism working; the defect is what it was for.

Back to. Main Index > Specification / Architecture Sub-Index

Revision note, v1.2: §12.5 and §13 updated for MPLPB-TEST-014 v2, which withdraws the v1 run as uninterpretable and reports a graded result in its place. §10 now defers to MPLPB-DEPLOY-015 for deployment settings rather than describing constituencies in prose only. No claim in v1 or v1.1 is withdrawn.

Revision note, v1.1: §12.5 and §13 amended to record MPLPB-TEST-014, a partial run of the ablation's mechanical half. The test remains open; the amendment exists because a paper that names a test as open while a run of part of it sits in the same repository is misdescribing its own evidence. No claim in v1 is withdrawn, and the §9 argument is untouched.

Revision note, v1: First issue. No supersession.

Three things are deliberately absent and should not be read as oversights. There is no distributed consensus mechanism, for the reason in §4.4. There is no content-level validation of taught pages, for the reason in §5.2. There is no threshold table in §8, for the reason stated there — a number invented at specification time would carry an authority the corpus has not earned.